

Case Study 7: Minimizing Misclassification Cost with XGBoost and Neural Networks

Kristin Henderson

August 2, 2025

1 Introduction

This study focuses on minimizing the losses from incorrect predictions in a binary classification problem where false positives and false negatives carry different costs. The analysis compares two modeling approaches, XGBoost and a neural network. Custom decision thresholds and evaluation metrics are used to reflect the unequal costs of the two error types.

2 Data

2.1 Dataset

The dataset includes 160,000 records with 50 features and a binary target variable. Of the 50 features, 45 are numeric and 5 are categorical. The full list of features is summarized in Table 1. The feature names are generic and are all denoted by **x** followed by a number (e.g., **x0**, **x1**, ..., **x49**), representing the order in which the features appear in the dataset. No information about their meaning or source is provided. Based on their values, the categorical features appear to represent region, month, weekday, percentage, and dollar amount. All numeric features, including percentage and dollar amount, contain both negative and positive values.

Among the numeric features, **x37_dollar_amt** has the widest range (approximately -4700 to 5600), followed by **x46** (approximately -200 to 150). If left unscaled, high-magnitude features, especially the dollar amount feature, may dominate scale-sensitive models. In contrast, **x0** and **x45** have the narrowest ranges (approximately -2 to 2). The target variable is integer-coded as 1 (positive class) and 0 (negative class).

Table 1: Dataset Features and Inferred Categories

Description	Feature(s)
Numeric features	x0-x23 , x25-x28 , x31 , x33-x36 , x38-x49
Categorical features	
Region (America, Asia, Europe)	x24
Month (Jan-Dec)	x29
Weekday (Mon-Fri)	x30
Percent	x32
Dollar Amount	x37
Target class (0 = negative, 1 = positive)	y

The target variable is imbalanced, with 59.9% of observations belonging to the negative class and 40.1% to the positive class. The class distribution is shown in Figure 1.

2.2 Cleaning

The dollar amount and percent columns were converted to numeric after stripping the dollar and percent signs. Month and weekday names were standardized. A separate `month_num` column was created by converting month names to integers 1–12. The same process was used for weekdays, creating a `weekday_num` column with values 1–5. For each model, only the categorical or numeric version of month and weekday was used, not both. Region names were also standardized and retained as categorical variables.

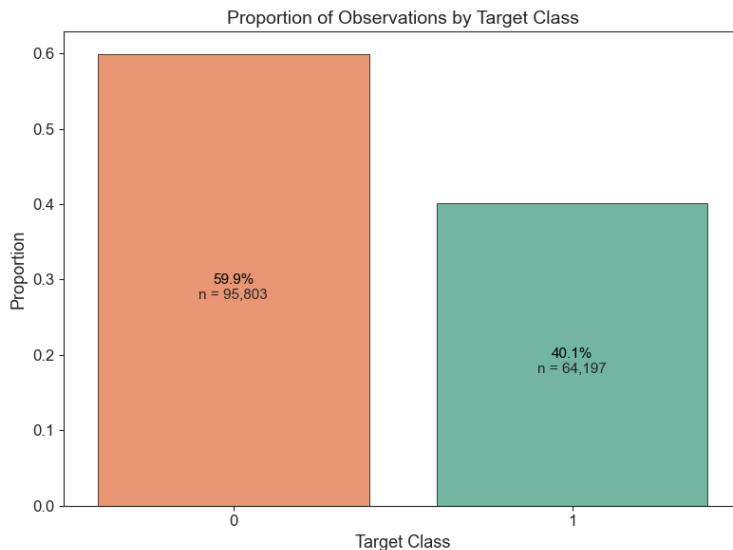


Figure 1: Proportion of observations in each target class, with counts and percentages labeled on the bars.

2.3 Missing Values and Imputation

All features contain between 21 and 47 missing values, accounting for fewer than 0.03% of each feature’s values. Two pairs of features are perfectly correlated: `x38` with `x41` and `x2` with `x6`.

There is no overlap in the rows with missing values between the correlated pairs. Therefore, `x41` and `x2` were imputed by modeling their linear relationships with their perfectly correlated counterparts. Specifically, `x41` was calculated as $1.1059 \times x38$, and `x2` as $0.6878 \times x6$. After imputation, `x38` and `x6` were removed from the dataset to avoid redundancy while preserving information.

The category distributions for region, month, and weekday are unbalanced. As shown in Figure 2, mid-week days and mid-year months are the most common. The categorical month and weekday variables were imputed with the mode, and the corresponding integer values were used to impute the numeric versions. Region was also imputed with the mode.

All numeric features approximately follow a normal distribution, as illustrated by representative examples in Figure 3. Therefore, the remaining missing values in numeric features were imputed using the column median.

3 Preprocessing

Both models used the same input data, but preprocessing steps differed based on the characteristics of the model type.

3.1 XGBoost Preprocessing

The integer-coded month and weekday features were used instead of one-hot encoding. Month values exhibited a monotonic increase in the positive class proportion, suggesting that the ordering was meaningful. In contrast, target class proportions were similar across weekdays showing no such trend. This suggests

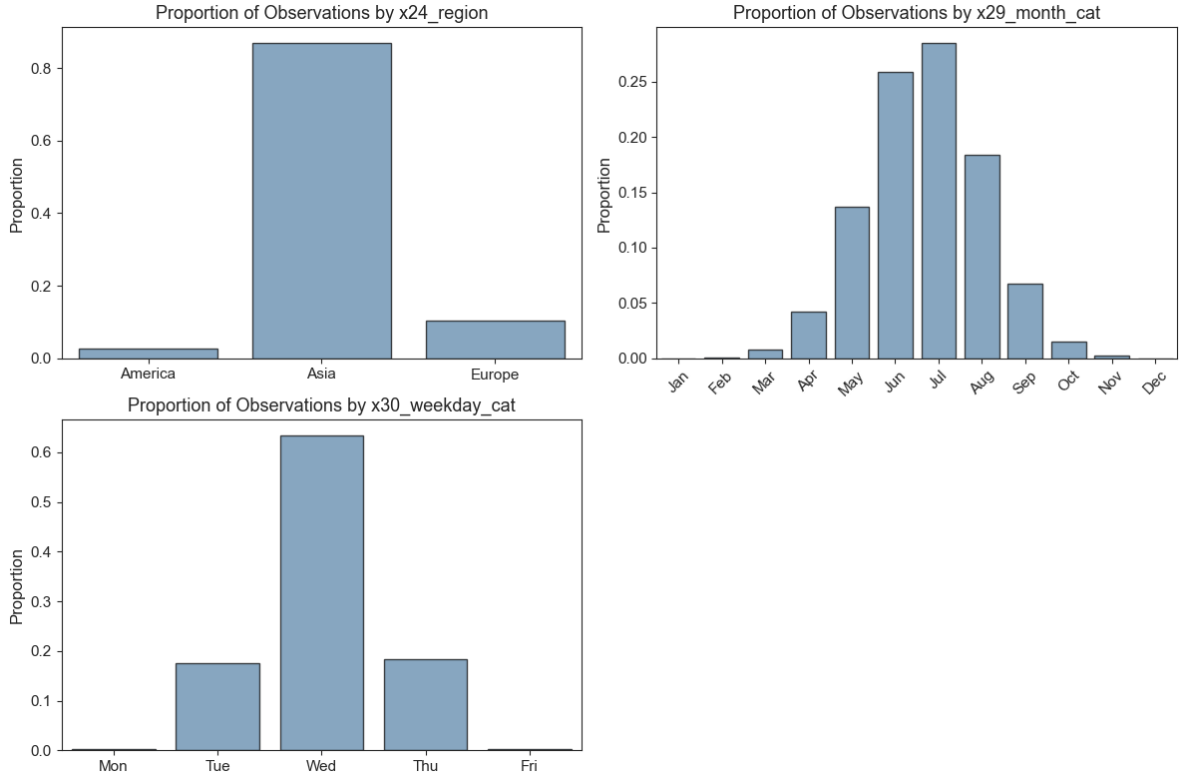


Figure 2: Proportions of observations in each category of the region, month, and weekday features.

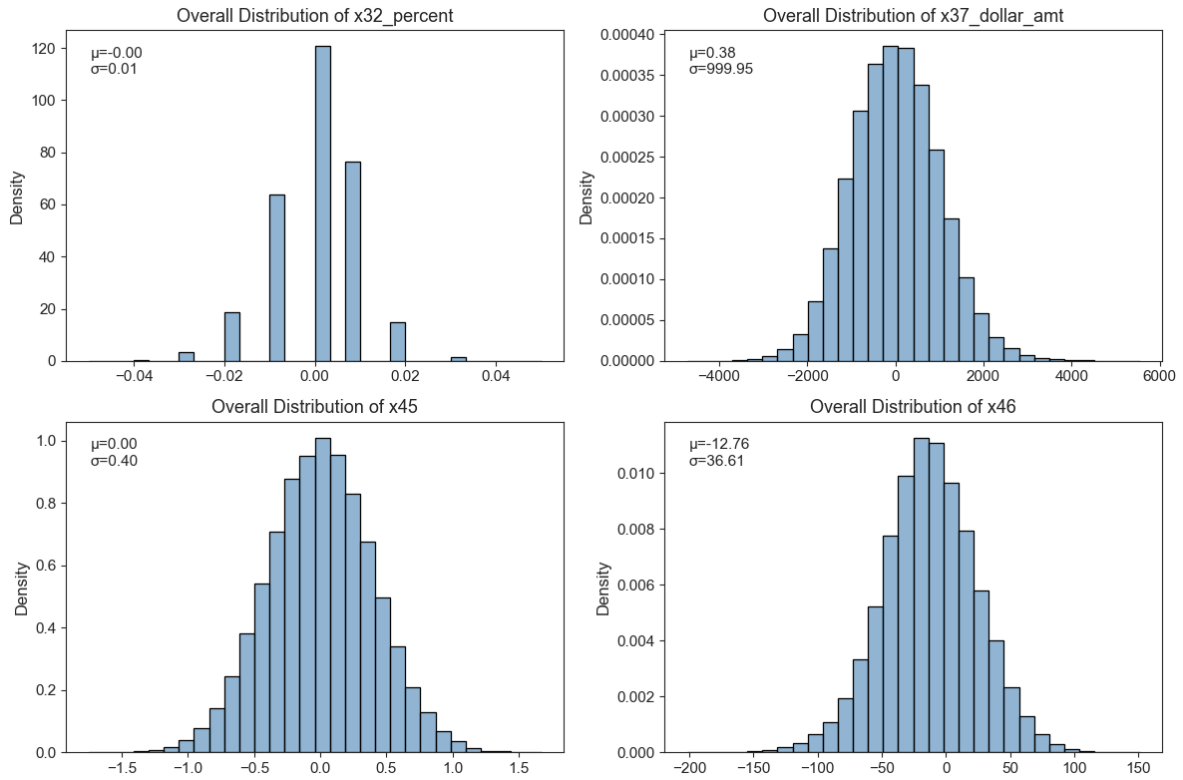


Figure 3: Histograms of representative numeric features: x32_percent, x37_dollar_amt, x45, and x46.

weekday is a nominal rather than an ordinal feature. However, since XGBoost uses threshold-based splits rather than interpreting numeric distances, the integer coding was retained for both. The region variable was one-hot encoded. The processed data was converted to a `DMatrix` for model training.

3.2 Neural Network Preprocessing

For the neural network, the integer-coded month feature was also used. However, the weekday feature was treated as categorical and one-hot encoded to avoid implying numeric distance between categories. Region was also one-hot encoded. Because neural networks are sensitive to feature scales, `StandardScaler` was applied to all numeric features, including the integer-coded month. Data was converted to PyTorch tensors within a custom cross-validation loop.

4 Modeling

4.1 XGBoost

The XGBoost model was created using the `xgboost` library. Data was converted to `DMatrix` format prior to training. A `binary:logistic` objective function was used to optimize logistic loss during training. Early stopping was applied with a patience of 10 rounds, and models were allowed up to 1,000 boosting rounds. Model performance and early stopping were evaluated based on the area under the receiver operating characteristic (ROC) curve (AUC).

A randomized hyperparameter search was performed using `xgboost.cv` with 50 parameter combinations and stratified five-fold cross-validation. Initially, `eta` (learning rate) was fixed at 0.3 but was later included in the tuning grid to slow convergence to within 200–500 rounds. The search space is shown in Table 2.

Table 2: XGBoost Hyperparameter Search Space

Hyperparameter	Values Considered
<code>max_depth</code>	15, 20, 25, 30, 35
<code>subsample</code>	Randomly sampled values between 0.5 and 1.0
<code>colsample_bytree</code>	Randomly sampled values between 0.5 and 1.0
<code>colsample_bylevel</code>	Randomly sampled values between 0.5 and 1.0
<code>colsample_bynode</code>	Randomly sampled values between 0.5 and 1.0
<code>reg_lambda</code>	0.01, 0.1, 1, 10, 100
<code>eta</code> (learning rate)	0.05, 0.1, 0.2, 0.3

The final model was trained using `xgboost.train` in a manual five-fold cross-validation loop with early stopping, generating out-of-fold predicted probabilities for the entire dataset. These predictions were used to calculate overall performance metrics and to tune the classification threshold.

4.2 Neural Net

The neural network was constructed using PyTorch Lightning with a feedforward architecture of five fully connected hidden layers, each with 1,000 neurons. The model accepted 54 input features after one-hot encoding or scaling, and used either a Gaussian Error Linear Unit (GELU) or Sigmoid Linear Unit (SiLU) activation function. Both activation functions promote stable decreases in loss during training and help avoid issues like vanishing gradients. They allow small negative values and perform well in deep networks, but use different formulas to shape their response curves. A single output node with sigmoid activation produced class probabilities.

Models were trained using the Adam optimizer with binary cross-entropy loss. Early stopping was triggered after five consecutive epochs without improvement in validation AUC (`patience=5`, `min_delta=0`). Logging was handled using CSV loggers, which tracked loss and AUC per epoch, and checkpoints were saved automatically.

A grid search was performed over combinations of dropout rate (0.2, 0.5), activation function (GELU, SiLU), batch size (1,024, 2,056), and learning rate (5e-5, 1e-4, 5e-4). For each configuration, five-fold stratified cross-validation was conducted using PyTorch tensors and DataLoaders. The training set used `shuffle=True` to promote generalization by presenting the model with data in a different order each epoch. The validation set used `shuffle=False` to ensure that evaluation remained consistent across epochs. The tuning process retained the best-performing model checkpoint per configuration. The hyperparameter search space is shown in Table 3.

Table 3: Neural Net Hyperparameter Search Space

Hyperparameter	Values Considered
Dropout	0.2, 0.5
Activation function	GELU, SiLU
Batch size	1,024, 2,056
Learning rate	5e-5, 1e-4, 5e-4

The final neural network model was selected based on the average validation AUC across a five-fold cross-validation run using early stopping. Out-of-fold predicted probabilities and performance metrics were computed using the stored models in evaluation mode (`model.eval()` with `torch.no_grad()`). These out-of-fold predictions were then used for final evaluation and threshold tuning.

4.3 Tuned Parameters

The best parameter combinations for each model are listed in Table 4.

The XGBoost model performed best with random subsampling at high proportions of rows and features: 99.1% of rows, 74.7% of features per tree, 86.0% per level, and 83.9% per node. This randomness helps reduce overfitting and improve generalization. Additional overfitting control came from a maximum tree depth of 25 and an L2 regularization term (λ) of 0.01. A learning rate of 0.1 (lower than the default 0.3) led to more gradual convergence, resulting in 212–276 boosting rounds per fold.

The neural net performed best with SiLU activation, a batch size of 2,056, and 50% dropout regularization applied to each hidden layer. The higher dropout rate helped reduce overfitting and improve generalization. A learning rate of 0.0001 led to convergence within 131–157 epochs per fold. Although all hyperparameter combinations performed similarly, the dropout rate and activation function appeared to have slightly greater influence on performance than the learning rate or batch size. The top configurations consistently used the 0.5 dropout rate, and six of the top seven used the SiLU activation function. Differences between batch sizes of 1,024 and 2,056 were negligible.

Table 4: Best Hyperparameter Combinations for Each Model

XGBoost	Neural Net
max_depth = 25	dropout = 0.5
subsample = 0.9911	activation = SiLU
colsample_bytree = 0.7473	batch size = 2,056
colsample_bylevel = 0.8595	lr (learning rate) = 0.0001
colsample_bynode = 0.8393	
reg_lambda = 0.01	
eta (learning rate) = 0.1	

Note: XGBoost parameter values have been rounded to four decimal places for clarity.

5 Results

5.1 Threshold Tuning

The best decision threshold for each model was determined by minimizing the total cost of misclassification, as shown in Figure 4. This cost was computed by summing \$27 for each false positive and \$13 for each false negative, and comparing the total cost across all threshold values. The higher cost for false positives reflects their greater impact within the domain.

For the XGBoost model, the threshold that minimized total cost was 0.663, resulting in a total misclassification cost of \$155,935. For the neural network, the optimal threshold was 0.699, resulting in a total cost of \$77,917. This amounts to a cost reduction of more than 50% compared to the XGBoost model.

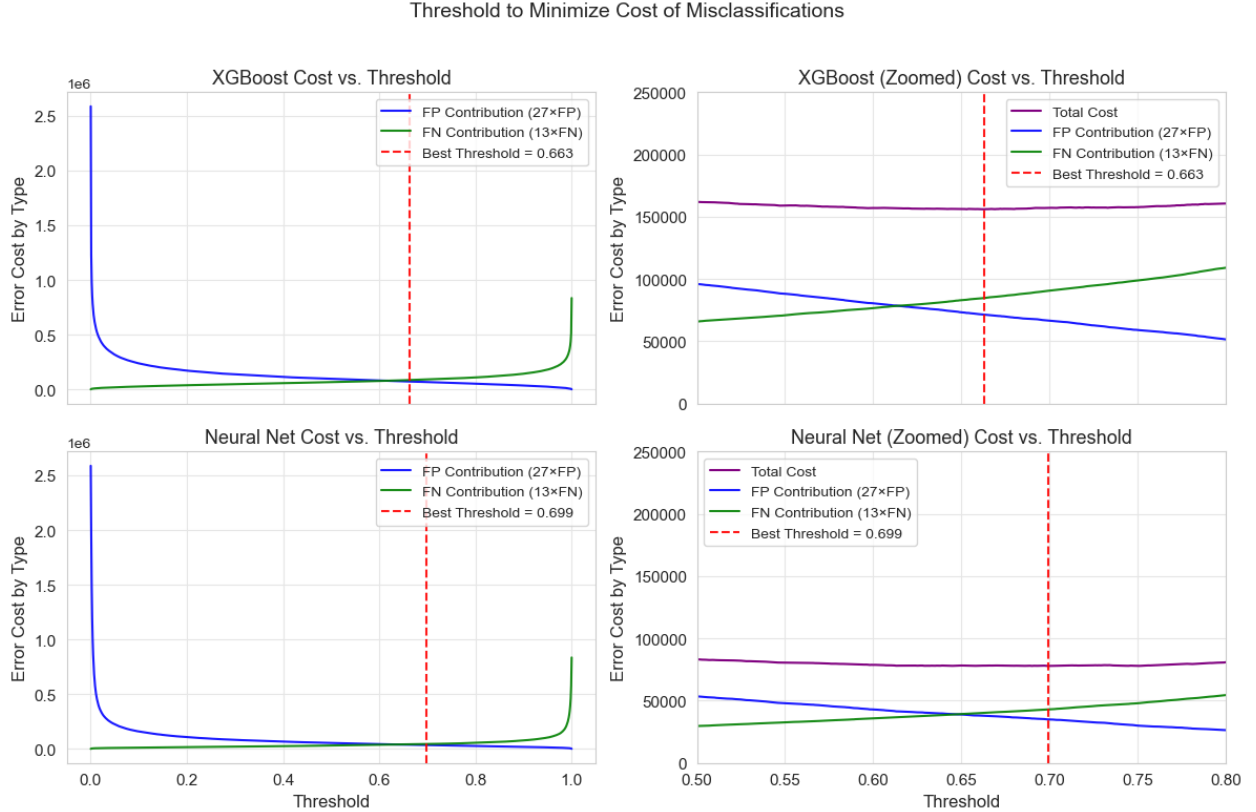


Figure 4: Cost curves for XGBoost (top row) and the neural network (bottom row), showing misclassification cost as a function of the decision threshold. The plots on the left display the separate costs of false positives (FP, blue) and false negatives (FN, green). The plots on the right show a zoomed-in view of FP, FN, and the total misclassification cost (purple). The optimal threshold is indicated in red.

5.2 Confusion Matrices

To generate the confusion matrices shown in Figure 5, the optimal classification threshold was used for each model: 0.663 for XGBoost and 0.699 for the neural network.

The neural network produced more true positives and true negatives, with fewer false positives and false negatives than the XGBoost model. Its greatest improvement was in the true positive rate, which increased by 5% over the XGBoost model. It correctly classified 3,210 more positive instances (5.0% of the positive class) and 1,344 more negative instances (1.4% of the negative class).

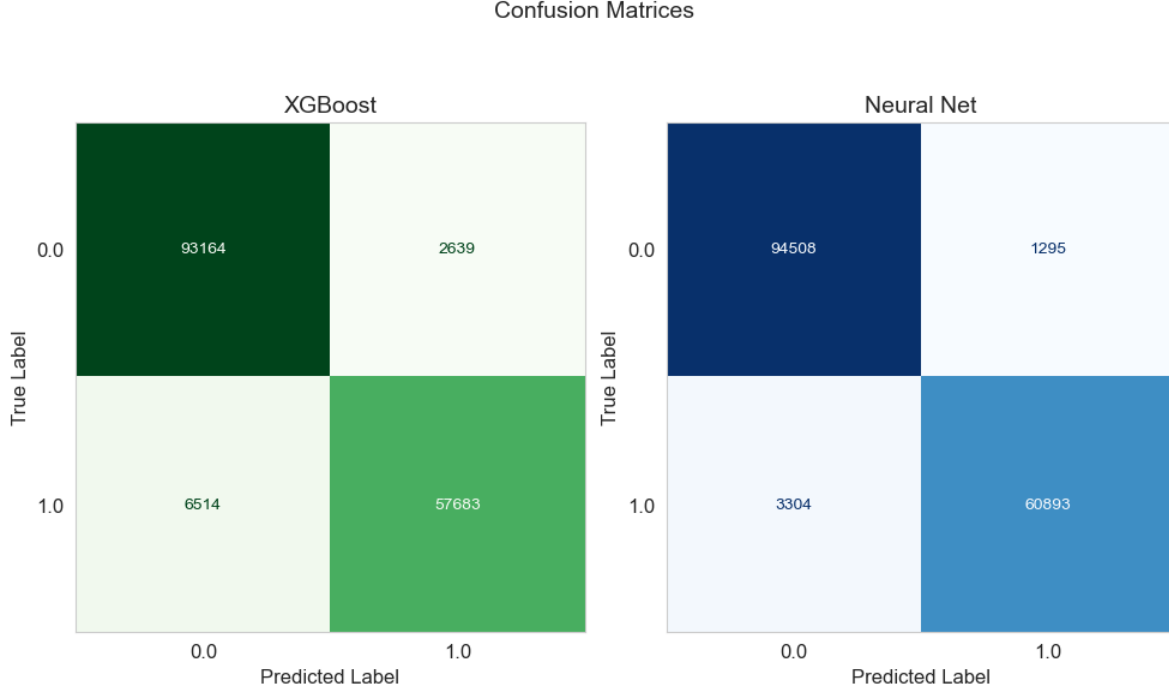


Figure 5: Confusion matrices showing classification results from out-of-fold predictions for the XGBoost model (green, left) and the neural network (blue, right). The optimal classification threshold was used for each model. True labels are shown on the vertical axis, and predicted labels on the horizontal axis. Each cell reports the number of instances classified into each outcome category.

5.3 Overall Metrics

All evaluation metrics, including AUC, accuracy, weighted precision, recall, and F1-score, were higher for the neural network than for the XGBoost model, as shown in Table 5. The neural network achieved an accuracy of 97.1% compared to 94.3% for the XGBoost model.

Because false positives are more costly, higher precision reflects better control over this error type. The neural network demonstrated this with a weighted precision of 97.1% and 1,295 false positives, compared to 94.3% and 2,639 false positives for the XGBoost model. Since the target classes were imbalanced, weighted metrics are reported. However, macro and weighted averages differed by less than 0.01, indicating consistent performance across classes.

Despite its stronger performance, the neural network required significantly more computation time. Training and prediction across five cross-validation folds took 27.3 minutes, compared to only 0.56 minutes for the XGBoost model.

Table 5: Overall Metrics on the Out-of-Fold Predictions and Cross-Validation Time by Model

Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1	AUC	CV Time (min)
XGBoost	0.9428	0.9433	0.9428	0.9425	0.9849	0.56
Neural Net	0.9713	0.9714	0.9713	0.9712	0.9925	27.3

Note: Weighted metrics are reported due to class imbalance and cost-based evaluation. However, macro and weighted metrics differed by less than 0.01.

5.4 Per-Class Metrics

As shown in Table 6, the largest performance differences between the models appear in the recall and F1 score for the positive class, and in the precision for the negative class. Recall for the positive class was 94.9% for the neural network compared to 89.9% for the XGBoost model, representing a 5 percentage point improvement in correctly identifying positive cases. This higher recall also contributed to a higher F1 score for the neural network on the positive class. Additionally, the neural network achieved a 3.1 percentage point increase in precision for the negative class, also known as the negative predictive value, reflecting fewer false negatives and more true negatives.

Table 6: Per-Class Classification Metrics on the Out-of-Fold Predictions for Each Model

Model	Class	Precision	Recall	F1-score	Support
XGBoost	0 (negative class)	0.9346	0.9725	0.9532	95,803
	1 (positive class)	0.9563	0.8985	0.9265	64,197
Neural Net	0 (negative class)	0.9662	0.9865	0.9762	95,803
	1 (positive class)	0.9792	0.9485	0.9636	64,197

6 Conclusion

The neural network model successfully achieved the goal of minimizing total misclassification cost, resulting in a cost of \$77,917. This is just over a 50% reduction compared to the XGBoost model’s cost of \$155,935. However, this improvement came at a significant computational expense. The neural network required 27.3 minutes to train and predict across five folds compared to 0.56 minutes for the XGBoost model. The largest performance difference between models was a five percentage point increase in the true positive rate for the neural network. Still, both models achieved accuracy, weighted precision, and weighted recall above 94%.

If time, compute resources, or access to a GPU are limited, the XGBoost model is a competitive alternative. However, when the sole priority is minimizing the financial cost of misclassification, the neural network is the best choice.